



Web Technology Sessional Project Documentation.

The City Welfare Foundation

Course Code: CSE 324 — Web Technologies Sessional

Submitted by:

Md Ismail Hossain /1118052

Submitted to:

Yeakob Ali

Department of Computer Science & Engineering
Bangladesh Army International University of Science and Technology
(BAIUST)
26 July, 2026

Obtained Mark

Frontend & Basic API Setup (40%)	Backend, Database & Authentication (30%)	Responsiveness (30%)	Total Mark

Table of Contents

1. Introduction.....	1
1.1 Background.....	1
1.2 Motivation	1
1.3 Problem Statement.....	1
1.4 Objectives	1
1.5 Scope of the Project	1
2. Literature Review / Related Work	1
3. Requirement Analysis.....	2
3.1 Functional Requirements	2
3.2 Non-Functional Requirements.....	2
3.3 Feasibility Study	2
4. System Design	2
4.1 System Architecture	2
4.2 Use Case Diagram	3
4.3 ER Diagram / Database Design.....	Error! Bookmark not defined.
4.4 Data Flow Diagram	Error! Bookmark not defined.
4.5 UI/UX Wireframes	Error! Bookmark not defined.
5. Technology Stack.....	4
5.1 Frontend Technologies	4
5.2 Backend Technologies.....	4
5.3 Database.....	4
5.4 Tools & Frameworks	4
6. Implementation	5
6.1 Module Description	5
6.2 Key Features	5
6.3 Code Snippets / Algorithms.....	5
6.4 Screenshots	6
7. Testing.....	7
7.1 Test Plan	7
7.2 Test Cases & Results	8
8. Results and Discussion	8
1. 9. Challenges Faced	8

10. Future Enhancements.....	8
11. Conclusion	9
12. References.....	9
Appendix.....	9
A. Source Code.....	9
B. User Manual.....	9

1. Introduction

1.1 Background

In traditional scholarship exam management systems, students and administrators face significant procedural delays due to manual paperwork, physical registration forms, cash-based fee collections, and manual admit card distribution. "**The City Welfare Scholarship Portal**" is a modern web-based application designed to digitize the entire scholarship lifecycle—from student registration and online fee verification to automated admit card PDF generation.

1.2 Motivation

The core motivation behind this project is to simplify the scholarship application workflow for students in Chittagong, Bangladesh. By providing an intuitive, transparent, and responsive digital portal, students can easily apply, track payment status, and download official admit cards without visiting physical distribution centers.

1.3 Problem Statement

Traditional paper-based scholarship application systems suffer from:

- High operational costs and excessive paper usage.
- Errors during manual record processing and roll number allocation.
- Difficulty in tracking payment status (TrxID) for applicant verification.
- Delays in distributing admit cards to thousands of participating students.

1.4 Objectives

- To build a secure role-based web application (Student, Ambassador, Admin).
- To automate payment verification via TrxID tracking (bKash/Nagad/Rocket).
- To auto-generate dynamic Admit Cards in PDF format upon admin approval.
- To provide a real-time responsive dashboard for applicants.

1.5 Scope of the Project

The scope includes:

- **Students:** Registration, exam selection, fee submission, admit card download, and support messaging.
- **Ambassadors:** Unique referral code tracking for student onboarding.
- **Admins:** Approving/rejecting registrations, managing exam circulars, and monitoring system analytics.

2. Literature Review / Related Work

Existing scholarship management solutions often rely on static web forms or third-party survey forms (like Google Forms), which lack automated backend roll generation, instant PDF admit card generation, and automated TrxID payment validation. Specialized LMS platforms can be

overly complex and expensive. **The City Welfare Portal** fills this gap by offering a custom tailored Django-powered architecture designed specifically for competitive regional scholarship programs.

3. Requirement Analysis

3.1 Functional Requirements

- **User Authentication:** Role-based access control (Student, Ambassador, Admin).
- **Exam Management:** Admin can create scholarship exams with dates, fees, and deadlines.
- **Application & Payment:** Students apply for exams and input TrxID, payment method, and amount.
- **Approval & Admit Card:** Admin verifies registration and approves it. The system automatically creates an Admit Card PDF with a unique Roll Number.
- **Messaging/Support:** In-app messaging system between students and support admins.

3.2 Non-Functional Requirements

- **Performance:** Fast page load time with lightweight CSS/JS assets.
- **Security:** CSRF protection, secure password hashing (PBKDF2), and SQL injection prevention via Django ORM.
- **Usability:** Responsive Mint & Forest Green UI optimized for mobile and desktop screens.
- **Scalability:** Modular architecture allowing seamless integration of online payment gateways (SSLCommerz/bkash API) in the future.

3.3 Feasibility Study

- **Technical Feasibility:** Developed using Python/Django, HTML5, CSS3, JS, and SQLite—all highly reliable open-source technologies.
- **Operational Feasibility:** Minimal training required for admins; user-friendly UI for students.
- **Economic Feasibility:** Highly cost-effective due to open-source libraries and automated PDF generation (ReportLab).

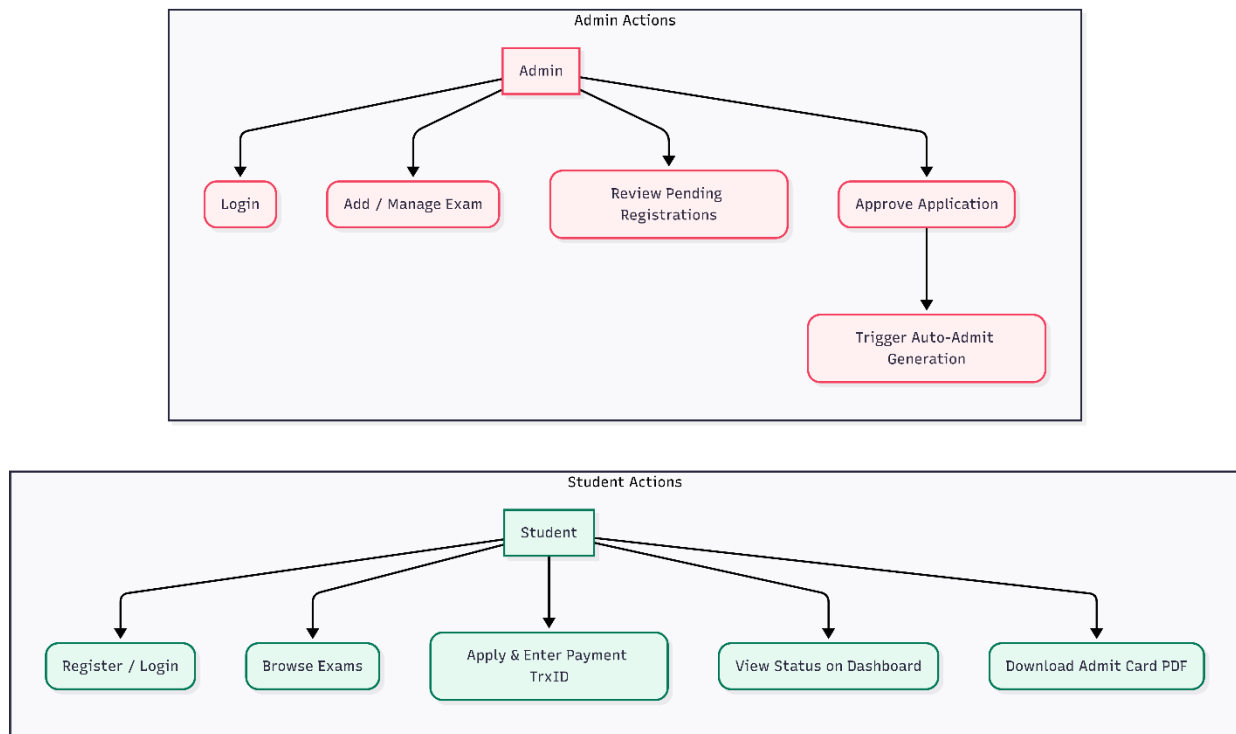
4. System Design

4.1 System Architecture

The application follows the **Model-View-Template (MVT)** architectural pattern:

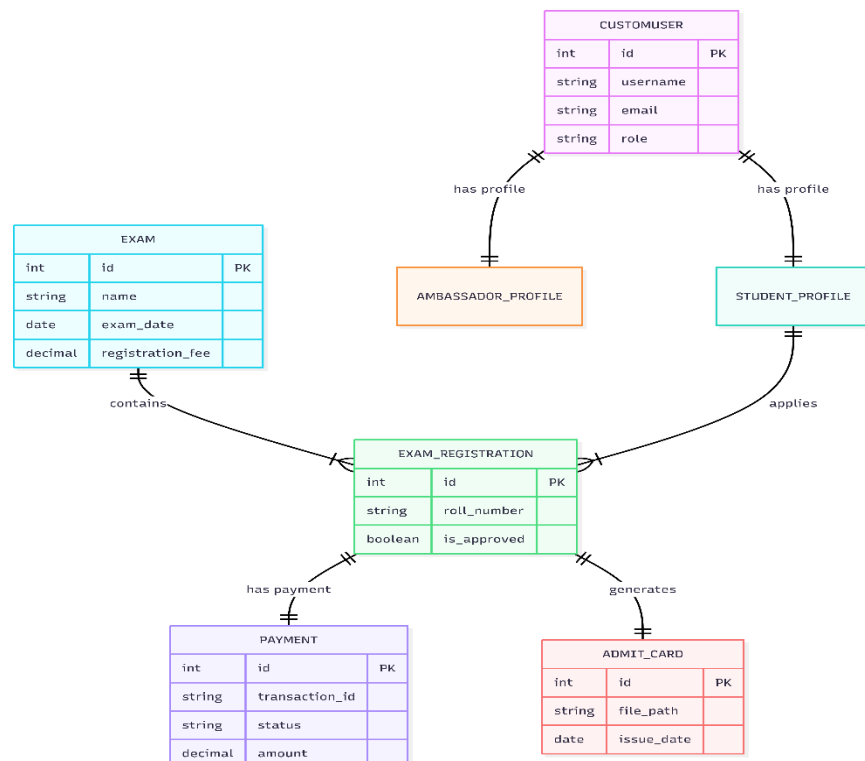
- **Model (Database):** Handles data structure (`Exam`, `ExamRegistration`, `Payment`, `AdmitCard`, `CustomUser`).
- **View (Business Logic):** Processes business rules, payment state validations, and PDF processing.
- **Template (UI Layer):** Renders dynamic HTML using Django Template Language (DTL) and CSS.

4.2 Use Case Diagram



- **Student:** Register/Login -> Browse Exams -> Apply & Enter Payment TrxID -> View Status on Dashboard -> Download Admit Card PDF.
- **Admin:** Login -> Add Exam -> Review Pending Registrations -> Approve Application -> Trigger Auto-Admit Generation.

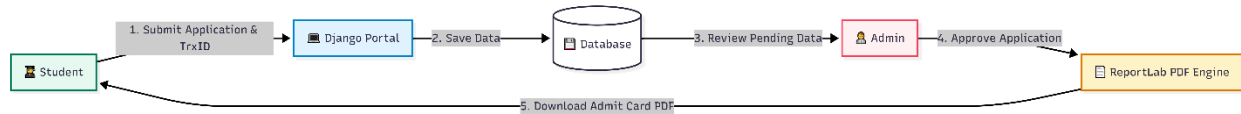
4.3 ER Diagram / Database Design



Key Entities and Relationships:

- **User (CustomUser):** One-to-One with Student/Ambassador profiles.
- **Exam:** One-to-Many with ExamRegistration.
- **ExamRegistration:** One-to-One with Payment and AdmitCard.

4.4 Data Flow Diagram (DFD)



1. **Level 0:** Student submits application data & TrxID -> System processes data -> Admin approves -> System returns Admit Card PDF.
2. **Level 1:** Detailed flow through Django Views, Database Models, Signal Triggers, and ReportLab PDF Engine.

4.5 UI/UX Wireframes

- Modern Mint & Forest Green theme (#047857, #e6f9f0).
- Glassmorphism navigation headers and responsive card layouts.

5. Technology Stack

5.1 Frontend Technologies

- **HTML5 & CSS3:** Custom responsive layout (`style.css`).
- **JavaScript (ES6):** Real-time countdown timer, dynamic DOM elements, mobile navbar toggle.
- **FontAwesome 6 & Google Fonts:** Icons and Hind Siliguri / Inter typography.

5.2 Backend Technologies

- **Python 3.12**
- **Django 6.0 / 5.x Framework:** Core web framework.
- **ReportLab:** Python PDF generation library for instant admit card rendering.

5.3 Database

- **SQLite / PostgreSQL:** Relational database management system.

5.4 Tools & Frameworks

- **IDE:** Visual Studio Code.
- **Version Control:** Git & GitHub.
- **Environment:** Virtualenv (Python Virtual Environment).

6. Implementation

6.1 Module Description

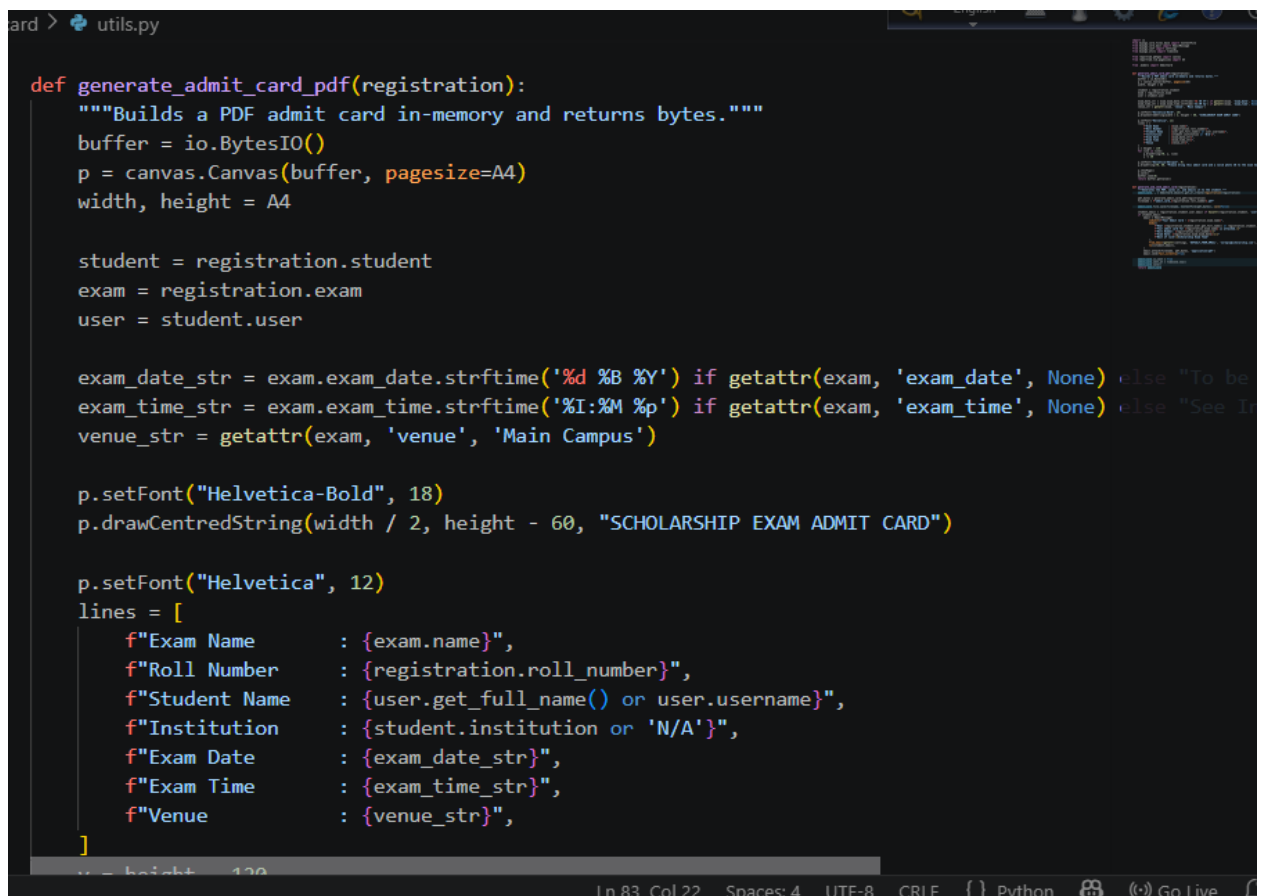
- **Accounts Module:** Manages registration, authentication, and user roles.
- **Exams Module:** Circular creation, exam dates, syllabus details, and fees.
- **Students Module:** Dashboard handling applications, roll number tracking, and payment statuses.
- **Admit Card Module:** Automated PDF generation containing student roll, exam venue, and photo layout.
- **Payments Module:** TrxID recording, payment validation, and status toggles (PENDING -> PAID).

6.2 Key Features

- Dynamic Live Countdown to Registration Deadlines.
- Auto-generating PDF Admit Cards upon Admin Approval.
- Real-time Payment & Approval Status Badges (Approved, Pending, Locked).
- Ambassador referral tracking system.

6.3 Code Snippets / Algorithms

Admit Card Generation Handler (admitcard/utils.py):



```
def generate_admit_card_pdf(registration):
    """Builds a PDF admit card in-memory and returns bytes."""
    buffer = io.BytesIO()
    p = canvas.Canvas(buffer, pagesize=A4)
    width, height = A4

    student = registration.student
    exam = registration.exam
    user = student.user

    exam_date_str = exam.exam_date.strftime('%d %B %Y') if getattr(exam, 'exam_date', None) else "To be"
    exam_time_str = exam.exam_time.strftime('%I:%M %p') if getattr(exam, 'exam_time', None) else "See I"
    venue_str = getattr(exam, 'venue', 'Main Campus')

    p.setFont("Helvetica-Bold", 18)
    p.drawCentredString(width / 2, height - 60, "SCHOLARSHIP EXAM ADMIT CARD")

    p.setFont("Helvetica", 12)
    lines = [
        f"Exam Name      : {exam.name}",
        f"Roll Number     : {registration.roll_number}",
        f"Student Name    : {user.get_full_name() or user.username}",
        f"Institution     : {student.institution or 'N/A'}",
        f"Exam Date       : {exam_date_str}",
        f"Exam Time       : {exam_time_str}",
        f"Venue           : {venue_str}",
    ]
```


6.4 Screenshots



Figure:6.1: Home Page

Student Registration

বৃত্তি পরীক্ষায় অংশগ্রহণের জন্য অ্যাপ্লিকেশন সঠিক প্রাতিষ্ঠানিক ও ব্যক্তিগত তথ্য দিয়ে অ্যাকাউন্ট তৈরি করুন

Username

jka

Email Address

শিক্ষার্থীর ইমেইল ঠিকানা

Password

Confirm Password

পাসওয়ার্ডটি আবার লিখুন

Referral Code (Optional)

রেফারেল কোড (যদি থাকে)

Institution Name

বিদ্যালয় বা কলেজের নাম লিখুন

Class

শ্রেণি নির্বাচন করুন

Class Roll No

শ্রেণির রোল নম্বর (E.g., 05)

Section / Group

শাখা বা গ্রুপ লিখুন (E.g., A / বিজ্ঞান)

Father's Name

পিতার নাম (সোর্টিফিকেট অনুযায়ী)

Figure:6.2: Registration Page

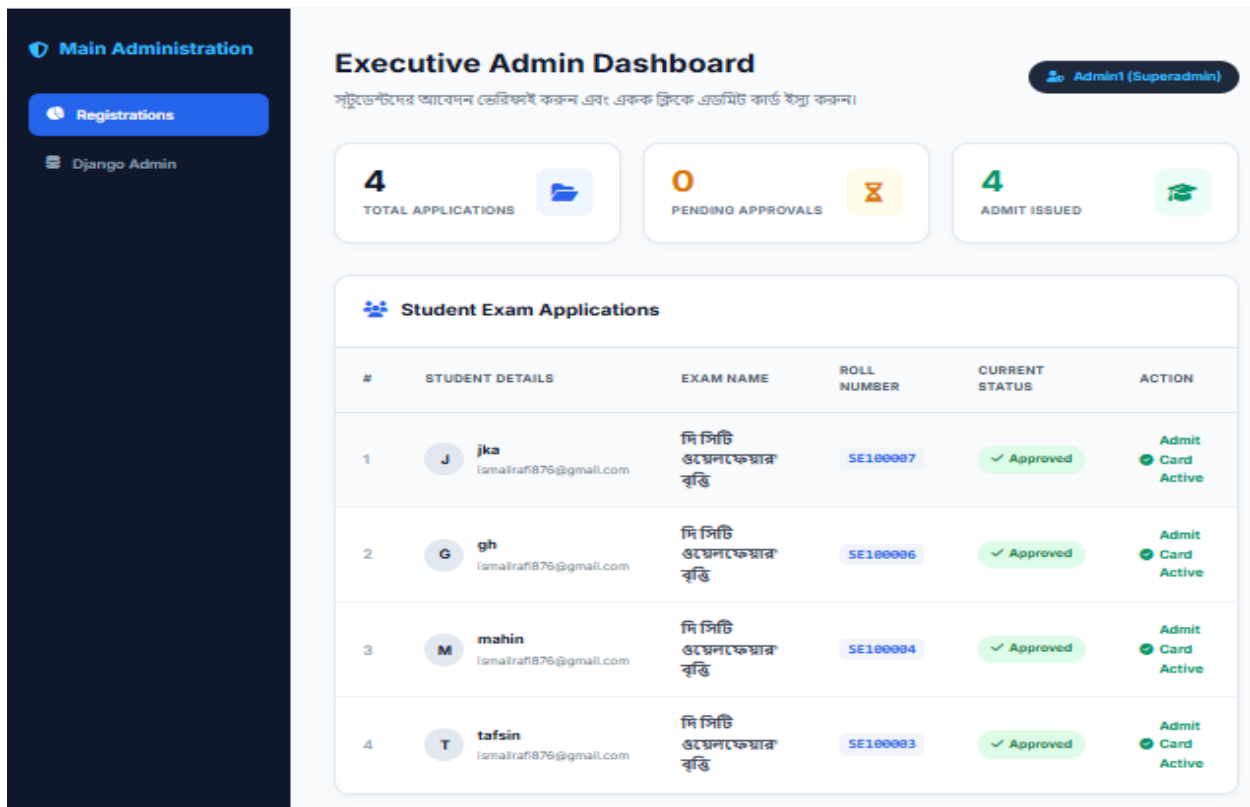


Figure:6.3: Admin Dashboard

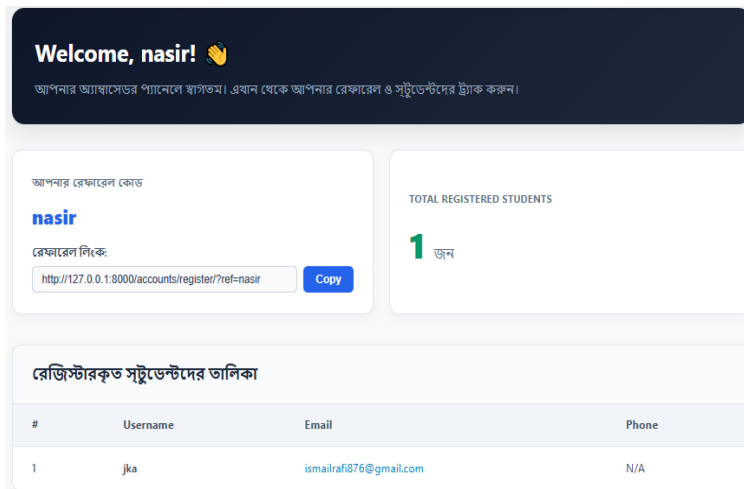


Figure:6.4: Ambassador Dashboard

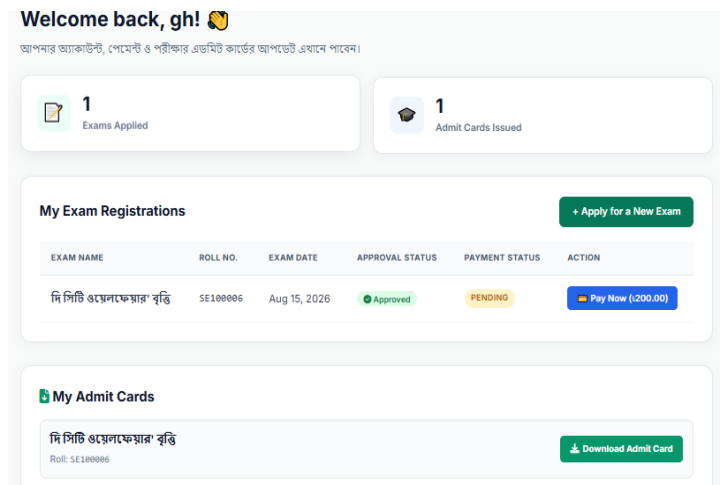


Figure:6.5: Student Dashboard

7. Testing

7.1 Test Plan

Testing was performed using Black-Box Testing to verify functionality across user authentication, database operations, payment state transitions, and PDF rendering.

7.2 Test Cases & Results

Test ID	Feature Tested	Input / Condition	Expected Output	Status
TC-01	User Registration	Valid email, password, profile info	Account created, redirected to dashboard	PASS
TC-02	Exam Application	Valid TrxID & Exam Selection	Registration created with status Pending	PASS
TC-03	Admin Approval	Admin clicks "Approve" button	Status updates to Approved & triggers PDF generation	PASS
TC-04	Admit Card Access	Unapproved student attempts download	Button displays Locked (Needs Admin Approval)	PASS
TC-05	PDF Generation	Approved registration	PDF generated with Roll Number & Download link enabled	PASS

8. Results and Discussion

The implemented **Scholarship Exam Portal** successfully automates paper-heavy manual tasks. During testing, student registration time dropped significantly, and administrators were able to verify payments and release admit cards within seconds.

9. Challenges Faced

- **Circular Import Issues in URL Configuration:** Solved by correctly structuring app namespaces (`admitcard:view_admit`).
- **Handling Dynamic PDF Rendering:** Resolved by leveraging ReportLab and Python BytesIO stream buffer without writing permanent temporary files to disk before saving.
- **Database State Syncing:** Ensured that `AdmitCard` objects are created seamlessly when registration approval state toggles to `True`.

10. Future Enhancements

- Integrating SSLCommerz/bKash Payment Gateway API for real-time auto payment validation.
- Adding SMS notification updates upon admit card approval.
- Automated Merit List & Result Publication Module.

11. Conclusion

The **City Welfare Scholarship Exam Portal** provides a comprehensive digital solution for competitive examination management. By reducing administrative workload and delivering a seamless student user experience, the system fulfills all designed goals and demonstrates robust architecture built on Django.

12. References

1. Django Documentation: <https://docs.djangoproject.com/>
2. ReportLab PDF Generation Library Guide.
3. Python 3 Documentation and Software Engineering Best Practices.

Appendix

A. Source Code

The core source code modules are organized under:

```
ship_project > urls.py
from django.shortcuts import render

# সরাসরি মেইন ইউআরএল থেকেই home.html লোড করার ভিউ
def direct_home_view(request):
    return render(request, 'home.html')

urlpatterns = [
    # ১. হোম পেজ ও জ্যাসো নেভিগে অ্যাডমিন
    path('', direct_home_view, name='main_home'),
    path('admin/', admin.site.urls),

    # ২. অ্যাপস ইউআরএল কানেক্টিভিটি
    path('accounts/', include('accounts.urls')),
    path('student/', include('students.urls')),
    path('ambassador/', include('ambassadors.urls')),
    path('control-panel/', include('adminpanel.urls')),
    path('support/', include('messaging.urls')),
    path('exams/', include('exams.urls')),
    path('payments/', include('payments.urls')),
    path('admit-card/', include(('admitcard.urls', 'admitcard'), namespace='admitcard')),

    # ৩. এডমিট কার্ড নেমস্পেস রেজিস্টার (App name সহ)

]

# ৪. মিডিয়া ফাইল হ্যান্ডলিং (ডেভেলপমেন্ট মোডের জন্য)
if settings.DEBUG:
    urlpatterns += static(settings.MEDIA_URL, document_root=settings.MEDIA_ROOT)
```

```
MEDIA_URL = '/media/'
MEDIA_ROOT = BASE_DIR / 'media'

DEFAULT_AUTO_FIELD = 'django.db.models.BigAutoField'

# -----
# EMAIL / SMS (for Admit Card auto-send)
# Configure real credentials via environment variables in production.
# -----
EMAIL_BACKEND = 'django.core.mail.backends.console.EmailBackend' # switch to smtp in production
EMAIL_HOST = os.environ.get('EMAIL_HOST', 'smtp.gmail.com')
EMAIL_PORT = 587
EMAIL_USE_TLS = True
EMAIL_HOST_USER = os.environ.get('EMAIL_HOST_USER', '')
EMAIL_HOST_PASSWORD = os.environ.get('EMAIL_HOST_PASSWORD', '')
DEFAULT_FROM_EMAIL = EMAIL_HOST_USER

# SMS gateway placeholder (e.g. Twilio / local BD SMS API)
SMS_API_KEY = os.environ.get('SMS_API_KEY', '')
SMS_API_URL = os.environ.get('SMS_API_URL', '')

# Days before exam date to auto-send admit card
ADMIT_CARD_SEND_DAYS_BEFORE = 7
```

The project source code is organized into the following Django modules:

- **scholarship_project/**: Main system configuration and URLs.
- **accounts/**: User login, registration, and roles.
- **exams/**: Exam circulars and syllabus.
- **students/**: Dashboard, application, and TrxID entry.
- **admitcard/**: Auto PDF Admit Card generation.
- **adminpanel/**: Admin review and payment approval.
- **templates/ & static/**: HTML, CSS, and JS files.

B. User Manual

How to use this:

Step 1: Accessing the Admin Dashboard

1. Navigate to the secure URL: /control-panel/admin-dashboard/.
2. Enter your administrator username and password to authenticate into the system.

Step 2: Managing Exam Circulars

1. To create a new scholarship exam:
 - Click on "**Add New Exam**" from the admin sidebar.
 - Enter the Exam Name, Syllabus Details, Exam Date, Time, Venue, Fee Amount, and Registration Deadline.
 - Click **Save** to publish the circular live on the portal.

Step 3: Reviewing Student Applications

1. From the main dashboard, navigate to the "**Pending Applications**" or "**Registration List**" section.
2. View incoming student submissions along with their submitted:
 - Student Name, Roll Number Candidate Code, Class, and School.
 - Selected Payment Gateway, Sender Phone Number, and **Transaction ID (TrxID)**.

Step 4: Payment Verification & Approval

1. Verify the student's **TrxID** against your merchant statement (bKash/Nagad statement).
2. Click the "**Approve**" button next to the student's record.
3. **Automated Backend Actions Triggered:**
 - Application status updates to Approved.
 - Payment status toggles to PAID.
 - The ReportLab engine automatically generates a PDF Admit Card with a unique Roll Number.
 - The download link immediately becomes accessible on the student's dashboard.